Faculty of Electrical Engineering, Mathematics and Computer Science

DQsym: A MATLAB/Simulink library for power system simulation using dynamic phasors

CONTENTS

1 DQsym Library Folder Structure **4**

1.1 Top-Level Structure 4

1.2 `#lib` Folder (Core Library) 4

1.3 Test Case Folders 5

1.3.1 Common Contents of a Test Case Folder 5

1.4 Typical User Workflow 6

1.5 checking the result 6

1.6 Development of New Test Cases 6

2 Developed test cases **7**

2.1 A simple RLC circuit 7

2.1.1 Basic Test Description 8

2.1.2 Parameters 8

2.1.3 MATLAB Structure 8

2.1.4 Run the Test Case and Results 9

2.2 MMC Station Connected to an AC Grid 9

2.2.1 Basic Test Description 9

2.2.2 Parameters 9

2.2.3 MATLAB Structure 10

2.2.4 Run the Test Case and Results 10

2.3 Point-to-Point HVDC System with MMC 11

2.3.1 Basic Test Description 11

2.3.2 Parameters 11

2.3.3 MATLAB Structure 13

2.3.4 Run the Test Case and Results 13

1

DQSYM LIBRARY FOLDER STRUCTURE

This section describes the organization and purpose of the `DQsym` library folder. The structure is designed to clearly separate the core library components from example and validation test cases.

1.1 Top-Level Structure

- `DP-GitHUB/`
  - `#lib/`
  - `RLC/`
  - `P2PHVDCMMc/`
  - `SingleMMc/`

1.2 `#lib` Folder (Core Library)

The `#lib` folder contains the main components of the `DQsym` library. This directory is the core of the project and includes all reusable models, functions, and supporting files required by the test cases as seen in Figure 1.2.1.

- All library blocks and utilities used by the `DQsym` framework reside here.
- This folder must be available on the MATLAB/Simulink path for any test case to run correctly.
- Users typically do not modify the contents of this folder unless extending or developing the library itself.

The source and grid interface blocks provide voltage or current inputs to the system in the standard *DQsym* dynamic phasor format, as defined in equation (1.2.1).The state-space block provides a standardised mechanism for implementing dynamic component models within the framework. The block represents converters, network elements, and control subsystems whose behaviour is governed by differential equation systems.

The mathematical operation blocks, multiplication, integration in DQsym provide reusable, parameterised components for implementing dynamic phasor equations using structured harmonic-domain signals. They support scalar, vector, and matrix operations—including harmonic-coupled multiplication and integration—while ensuring compatibility and modularity across network, converter, and control models. Measurement and post-processing blocks interface the internal DQsym representation with conventional time-domain signals, enabling waveform reconstruction, visualisation, and comparison with EMT simulations. Auxiliary utility blocks handle signal routing and interfacing with standard Simulink components, improving model clarity, scalability, and maintainability.

$$\begin{bmatrix} \langle x_{dq,p} \rangle_k \\ \langle x_{dq,n} \rangle_k \\ \langle x_{dq,z} \rangle_k \end{bmatrix} = \begin{bmatrix} \mathbf{x}_d^p(k) + j\mathbf{x}_q^p(k) \\ \mathbf{x}_d^n(k) + j\mathbf{x}_q^n(k) \\ \mathbf{x}_d^z(k) + j\mathbf{x}_q^z(k) \end{bmatrix} \tag{1.2.1}$$

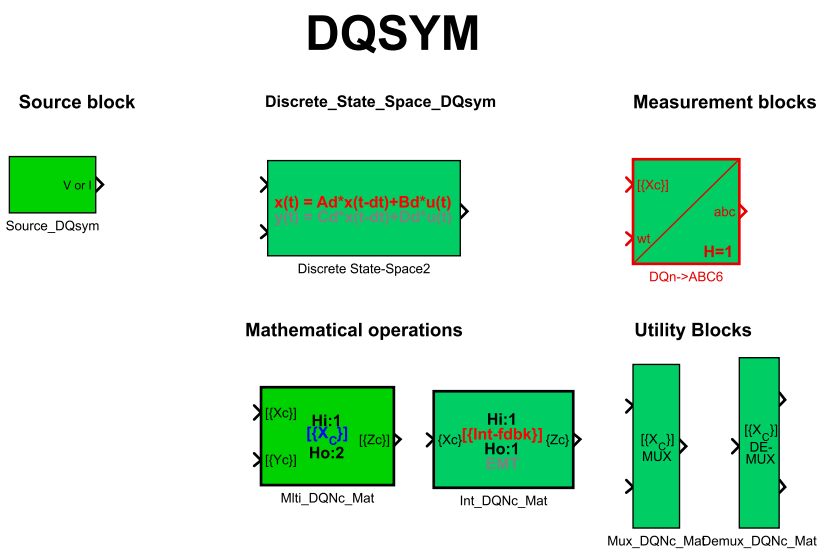


Figure 1.2.1: Overview of *DQsym* library in Matlab/Simulink.

1.3 Test Case Folders

The folders `P2PHVDCMMc` and `SingleMMc` are example and validation test cases built using:

- The `DQsym` library
- The *Specialized Power Systems* (SPS) library in Simulink

Each test case folder follows a similar internal structure.

1.3.1 Common Contents of a Test Case Folder

- `slprj/` Automatically generated Simulink cache and build files. This folder is created by Simulink during model execution and can be deleted and regenerated if needed.
- `*.slx` Simulink model files defining the test case topology. These files contain the electrical network, control systems, and interfaces built using the SPS library and `DQsym` components.

- `*.m` MATLAB script used to run the test case. This is the main file executed by the user. When run, the script:
  - Loads and initializes all required parameters
  - Configures the simulation environment
  - Opens and runs the corresponding `.slx` model
- `*.mat` MATLAB data files containing parameter sets, or precomputed variables required by the test case.

1.4 Typical User Workflow

1. Ensure the `#lib` folder and test case folder are added to the MATLAB path.
2. Navigate to the desired test case folder.
3. Open or directly run the associated `.m` file.
4. The script automatically loads parameters and executes the corresponding Simulink model.

This structure allows users to easily run, modify, and extend test cases while keeping the `DQsym` library modular and reusable.

1.5 checking the result

At this stage of development the reuslt where check for being overlapped with the reference MATLAB model.

1.6 Development of New Test Cases

When developing a new test case within the `DQsym` framework, the state–space representation of the system must be provided. In particular, the corresponding state–space matrices (e.g., **A**, **B**, **C**, and **D**) for the new case are required. These matrices are used by the `DQsym` library to define the dynamic behavior of the system and to ensure consistency between the analytical model and the Simulink implementation. The state–space matrices should be derived and validated prior to integration, and then incorporated into the parameter-loading scripts and Simulink models associated with the new test case.

State–Space and Input Definition in `DQsym` The state–space model of the new case must be entered directly into the State–Space block provided within the `DQsym` library. The previously derived state–space matrices are configured in this block to represent the system dynamics. In addition, all system inputs must be defined in the `DQsym` format. This requires using the dedicated input and interface blocks supplied in the library to ensure compatibility with the internal signal conventions, reference frames, and data structures used by `DQsym`.

2

DEVELOPED TEST CASES

2.1 A simple RLC circuit

This section describes a basic RLC test case implemented using the `DQsym` library. The test case is intended as an introductory example to demonstrate the modeling workflow, parameter definition, and simulation execution within the `DQsym` framework. The test case file is `DQsymRLC.slx`.

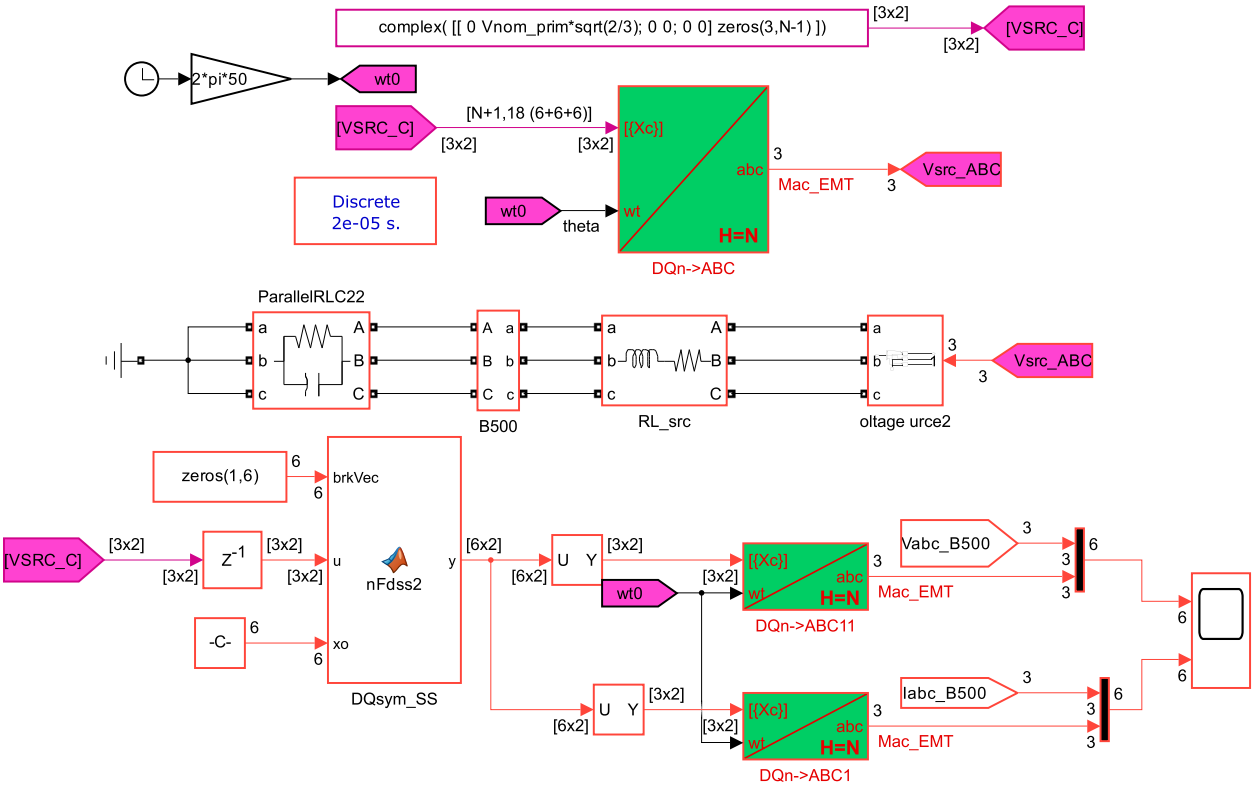


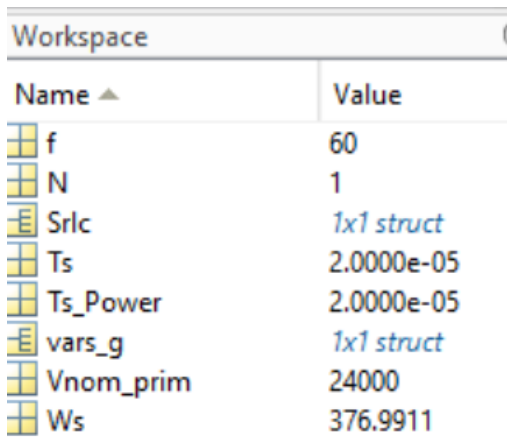
Figure 2.1.1: Overview of RLC test case in both the SPS and DQsym library blocks.

2.1.1 Basic Test Description

The RLC test case represents a simple linear electrical circuit composed of resistive, inductive, and capacitive elements. It is used to validate the correct integration of state–space models, input formatting, and signal handling in `DQsym`. The circuit topology is implemented in Simulink using blocks from the Specialized Power Systems library combined with `DQsym` components. The electrical network is interfaced with the `DQsym` State–Space block, which captures the dynamic behavior of the RLC circuit.

2.1.2 Parameters

All electrical parameters, including resistance, inductance, and capacitance values, are defined explicitly for the test case in `Srlc.mat`. These parameters are selected to ensure stable operation and clear observation of the system dynamics.



Workspace

Name	Value
f	60
N	1
Srlc	*1x1 struct*
Ts	2.0000e-05
Ts_Power	2.0000e-05
vars_g	*1x1 struct*
Vnom_prim	24000
Ws	376.9911

Figure 2.1.2: Test case parameters as in the workspace.

2.1.3 MATLAB Structure

The parameters and state–space matrices of the RLC circuit are organized in a MATLAB structure `Srlc.mat`. This structure contains the system matrices, initial conditions, and any additional configuration data required by the simulation.

A dedicated MATLAB `RLC.m` file serves as the main entry point for the test case. This script loads the parameter structure, initializes the simulation environment, and prepares the corresponding Simulink model for execution.

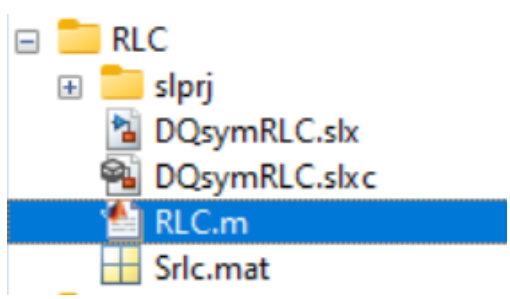


Figure 2.1.3: Test case folder tree.

Running the main MATLAB file (`RLC.m`) automatically compiles the test case by loading the Simulink model, assigning parameters to the `DQsym` blocks, and configuring simulation settings.

2.1.4 Run the Test Case and Results

Once complied, the test case is executed directly from the MATLAB script `RLC.m` with the following commands run: run('DQsymRLC') and sim('DQsymRLC'). Simulation results such as voltages, currents are generated and can be visualized using scopes or post-processing scripts to verify the expected dynamic response of the RLC circuit.

2.2 MMC Station Connected to an AC Grid

This section describes a test case of a Modular Multilevel Converter (MMC) station connected to an AC grid through a transformer. The test case demonstrates full control capability of the MMC to independently regulate active and reactive power using the `DQsym` library.

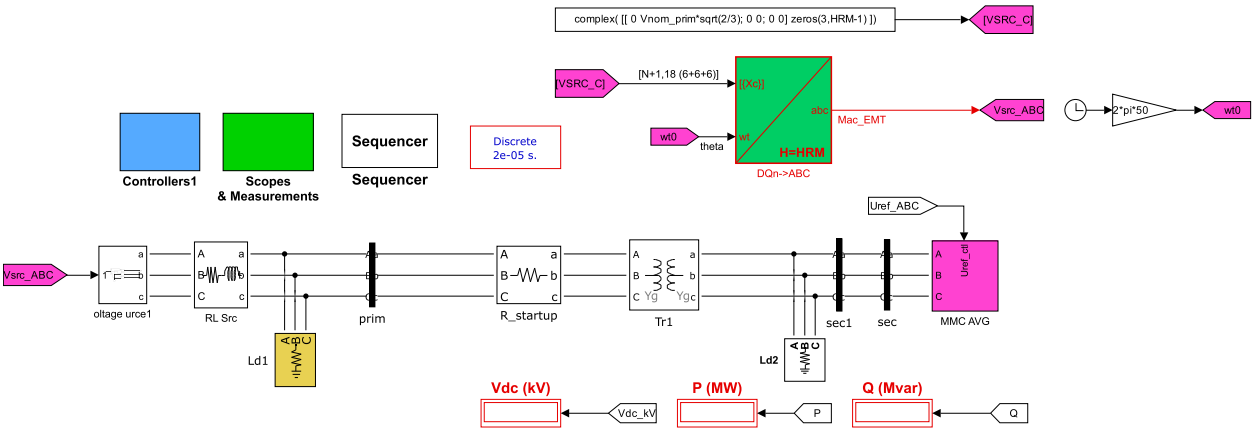


Figure 2.2.1: Overview of MMC station test case in both the SPS and DQsym library blocks.

2.2.1 Basic Test Description

The test case represents an MMC-based converter station interfacing an AC grid via a coupling transformer. The objective of the test is to validate the dynamic performance of the MMC model and its control system when regulating active and reactive power under grid-connected operation.

The system topology is implemented in Simulink using components from the Specialized Power Systems library and the `DQsym` library. It includes the MMC station, the grid-side transformer, and an equivalent AC grid model. The converter dynamics are represented using `DQsym` State–Space blocks, while the electrical interface and measurements are handled through dedicated `DQsym` input and output blocks.

2.2.2 Parameters

The test case parameters include MMC electrical characteristics (arm inductance, resistance, capacitance, and number of submodules), transformer parameters, and AC grid parameters such as grid voltage, frequency, and short-circuit strength. Control parameters for the

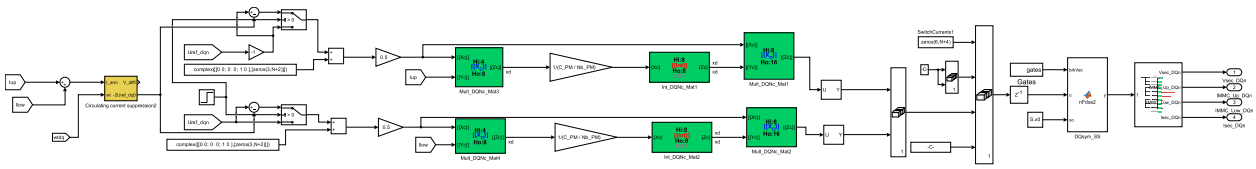


Figure 2.2.2: Upper and lower voltages as calculated using DQsym library blocks.

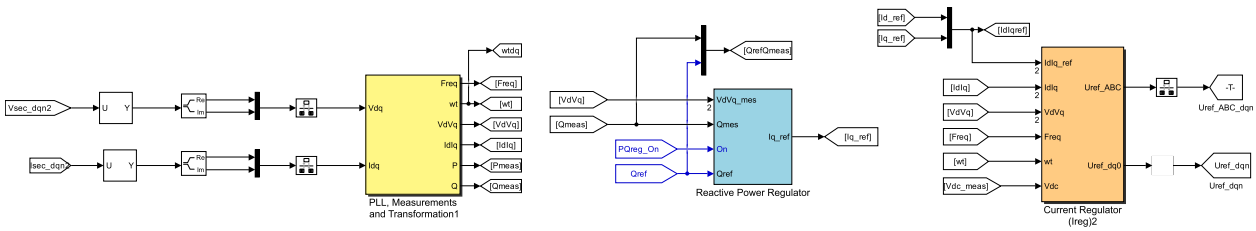


Figure 2.2.3: MMC single-station control structure.

inner current loops and outer active and reactive power control loops are also defined in `dqn_C_PQ_HVsMC.m` file and stored in `Sdq.mat`

2.2.3 MATLAB Structure

All system and control parameters, including the state–space matrices of the MMC model, are organized in a MATLAB structure `Sdq.mat`. This structure contains converter parameters, grid and transformer data, controller gains, and initial operating points required for simulation.

A MATLAB `dqn_C_PQ_HVsMC.m` file serves as the main execution script for the test case. This file loads the parameter structure, initializes the MMC, and configures the control references for active and reactive power before launching the Simulink model.

Executing the main MATLAB file `dqn_C_PQ_HVsMC.m` compiles the test case by loading the Simulink model, assigning parameters to the `DQsym` State–Space and control blocks, and configuring the simulation settings for grid-connected operation.

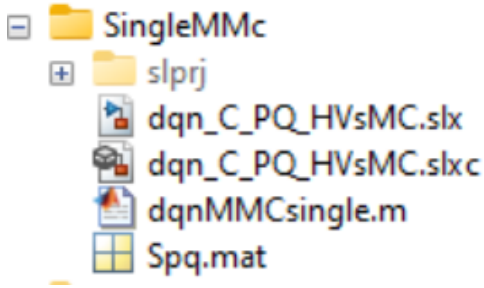


Figure 2.2.4: MMC single-station folder structure.

2.2.4 Run the Test Case and Results

After building the model `dqn_C_PQ_HVsMC.slx`, the simulation is executed from the main script using codes 1- open `dqn_C_PQ_HVsMC.slx` 2- sim `dqn_C_PQ_HVsMC.slx`. The results include AC-side voltages and currents, active and reactive power responses. These outputs are used to verify correct power control performance, steady-state tracking, and dynamic response of the MMC station connected to the AC grid.

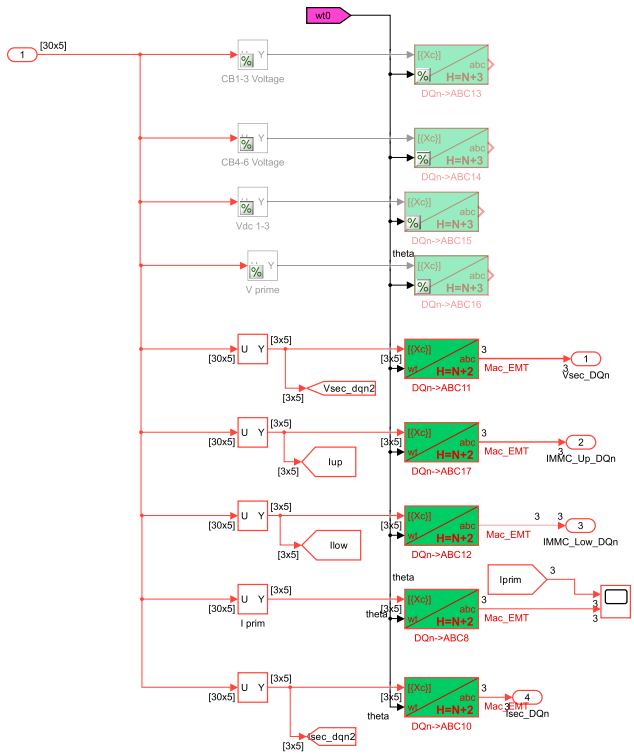


Figure 2.2.5: outputs transformed into three phase values to be compared with Matlab/Simulink model.

2.3 Point-to-Point HVDC System with MMC

This section describes a point-to-point HVDC transmission system based on Modular Multilevel Converters (MMCs). The test case demonstrates the dynamic behavior and coordinated control of two MMC stations connected through a DC link using the `DQsym` library.

2.3.1 Basic Test Description

The system topology is implemented in Simulink using the Specialized Power Systems library and `DQsym` components. Each MMC station is connected to its respective AC grid through a transformer. The two converters are interconnected via a DC transmission line or DC equivalent circuit. The converter dynamics are represented using `DQsym` State–Space blocks, with dedicated blocks for DQ-domain signal conversion and interfacing.

2.3.2 Parameters

The test case parameters are loaded using file `P2PHVDCMMC.m`. include MMC electrical parameters for both terminals (arm inductances, resistances, capacitances, number of submodules), DC link parameters (DC voltage rating, line resistance and inductance), transformer parameters, and AC grid characteristics at each end are all saved in `SFu.mat`. Control parameters for inner current controllers, DC voltage control, active power control, and reactive power control are also defined.

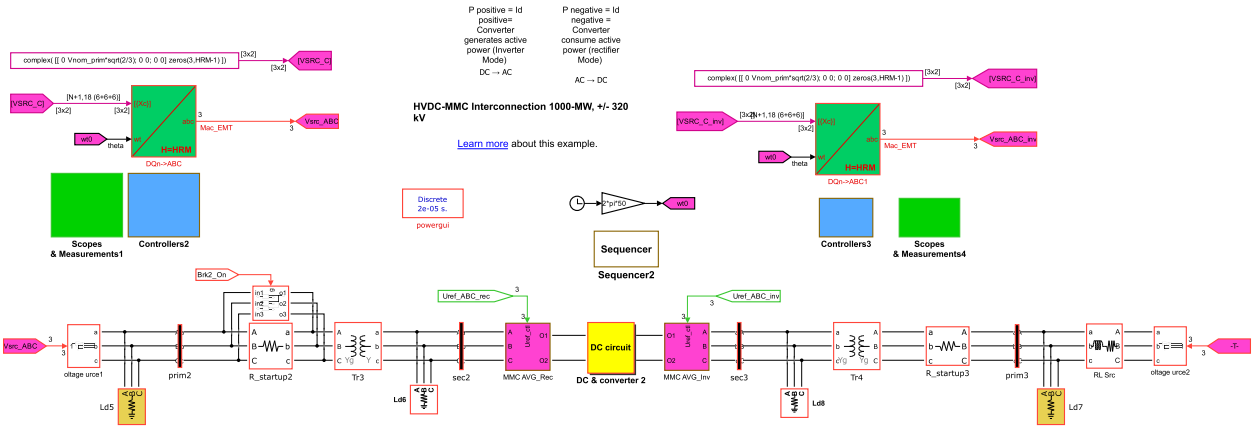


Figure 2.3.1: Overview of P2P test case simulation using *DQsym* library in Matlab/Simulink.

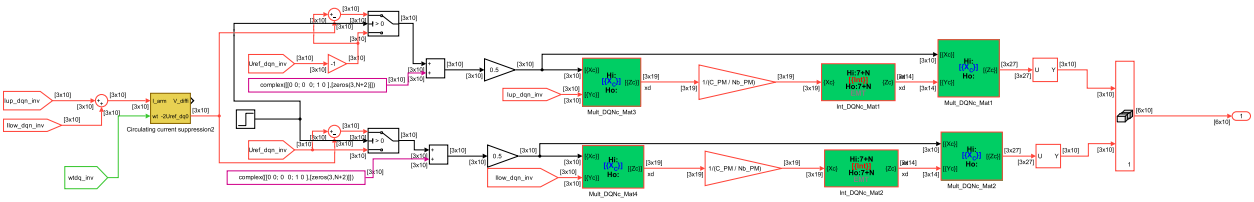


Figure 2.3.2: Overview of upper and lower arm current of the inverter MMc using *DQsym* library in Matlab/Simulink.

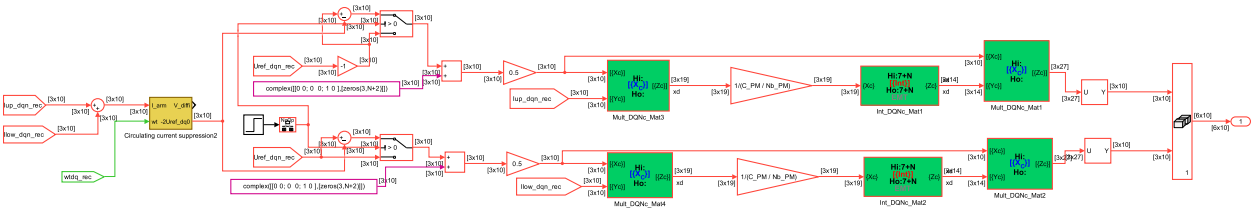


Figure 2.3.3: Overview of upper and lower arm current of the rectifier MMc using *DQsym* library in Matlab/Simulink.

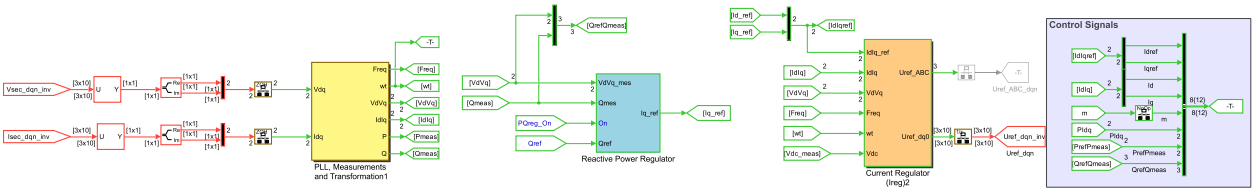


Figure 2.3.4: Overview of the inverter MMc control system using *DQsym* library in Matlab/Simulink.

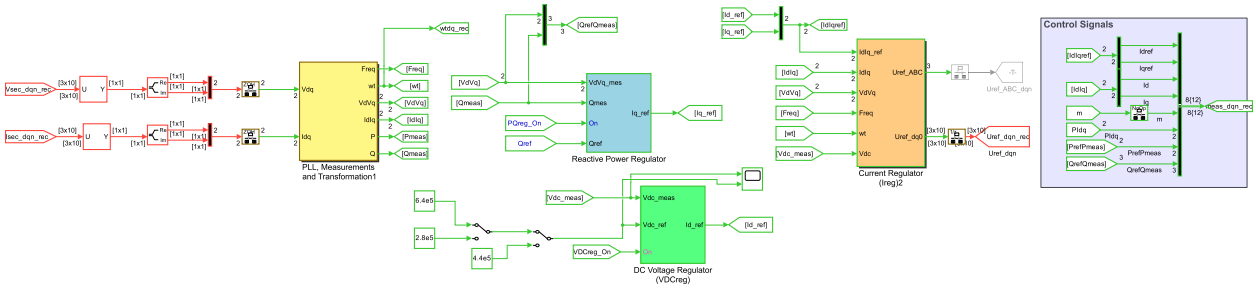


Figure 2.3.5: Overview of the rectifier MMc control system using *DQsym* library in Matlab/Simulink.

2.3.3 MATLAB Structure

All system, converter, and control parameters are organized in a structured MATLAB data object. This structure includes the state–space matrices for each MMC station, DC network parameters, controller gains, reference values, and initial operating conditions for both terminals.

A dedicated MATLAB `P2PHVDCMMC.m` file serves as the main execution script for the point-to-point HVDC test case. The script loads the parameter structures SFu.mat, assigns operating modes to each MMC station (e.g., DC voltage control or active power control), initializes references, and complies the Simulink model for simulation.

Executing the main MATLAB file complies the test case by loading the Simulink model, configuring the `DQsym` State–Space and control blocks for both MMC stations, and initializing the DC link and grid interface components.

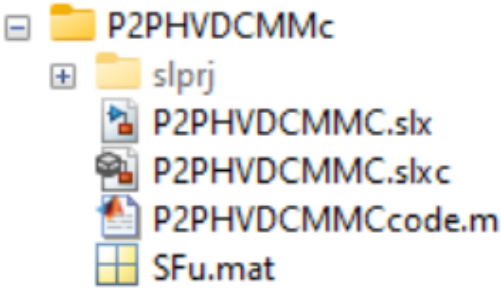


Figure 2.3.6: Test case folder structure.

2.3.4 Run the Test Case and Results

After compiling the model, the simulation is executed from the main script used codes 1- open P2PHVDCMMC.slx 2- sim P2PHVDCMMC.slx . The results include DC voltage and current waveforms, active and reactive power exchange at both AC terminals, and internal MMC state variables. These results are used to assess power flow control, DC voltage regulation, and the dynamic interaction between the two MMC stations in the HVDC system.

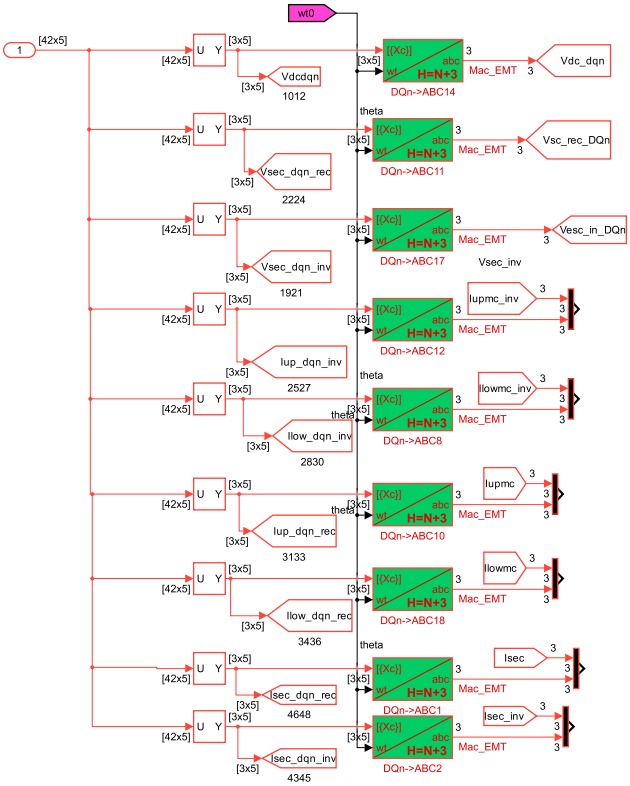


Figure 2.3.7: outputs transformed into three phase values to be compared with Matlab/Simulink model.